Fast functional mutation via ownership

The sealed discipline, the count of one, the examples, and the boundary

Gavin Mendel-Gleason
Scidonia

2026

1 The problem

Placeholder. The problem is stated here, in prose. This draft marks the space it takes and does not fill it; the case for the solution starts at section 3.

2 The solution

Placeholder. The solution is stated here, in prose, and the rest of the draft is the case for it: the discipline that establishes the condition (section 3), the count it produces in the model (section 4), the updates that work (section 5), and what is not claimed (section 6).

3 The sealed approach in Lean

A write needs exclusive access to the object it writes, and reads may be shared; the discipline that arranges this lives in the types and the proofs of the program, which the compiler erases. Its last step is a class whose instances supply almost nothing: a state type joins by naming the layout it is and proving that it is that layout's state type. The library defines the read, the write and the encoding of a read result from those two fields, so an instance author cannot choose any of the three, and a client of a sealed type has no operation that could hand back a mutable part of the state.

Definition 3.1 (sealed state variable)**.** An instance of `SealedStateVar` supplies a layout and an equality, and nothing else. Its read is the layout's read, its write is the layout's write, and its encoding of a read result is the library's encoding.

The sealed class *Scidonia/FracPerm/StateVar/Sealed.lean, lines 55–59*

```
class SealedStateVar (σ : Type) where
  layout : Layout
  rep : σ = State layout
```

The state types that join the class are the shapes a layout denotes: a leaf of a closed universe of observations, a binary product of two layouts — so arbitrarily many fields by nesting — or an array of states.

The shape of a state *Scidonia/FracPerm/StateVar/Layout5.lean, lines 122–136*

```
inductive Layout where
  | leaf (o : ObsDesc)
  | struct (left right : Layout)
  | arr (elem : Layout)
deriving DecidableEq, Repr

abbrev State : Layout → Type
  | .leaf o => ObsEl o
  | .struct a b => State a × State b
  | .arr a => Array (State a)
```

A read is a path whose end is a leaf of the observation universe. The proof field is what makes a read name a leaf and nothing else: a path that ends at a `.struct` or an `.arr` has no leaf description, so a whole-array read cannot be written down.

The read path *Scidonia/FracPerm/StateVar/Layout5.lean, lines 169–175*

```
structure ObsPath (d : Layout) where
  path : Path d
  obs : ObsDesc
  leaf : At path = .leaf obs
```

The write is one function at a path. It decides first whether the path addresses anything, rebuilds each struct cell on the path from its updated component, and, at an array element, takes the element out, checks it, updates it and puts it back: taking it out is what keeps the array from holding the element while the update below it runs.

The path-indexed write *Scidonia/FracPerm/StateVar/Layout5.lean, lines 778–782*

```
def setLeaf {d : Layout} (p : ObsPath d) (x : ObsEl p.obs) (s : State d) :
    State d × Bool :=
  if inBounds p.path s then
    (setAt p.path (cast (congrArg State p.leaf.symm) x) s, true)
  else (s, false)
```

The sealed surface is that write with the state transported into its layout and back, reporting what the layout's write reported.

The library's write of a sealed state *Scidonia/FracPerm/StateVar/Sealed.lean, lines 85–89*

```
def setLeaf {σ : Type} [SealedStateVar σ]
    (p : ObsPath (SealedStateVar.layout (σ := σ))) (x : ObsEl p.obs) : σ → σ × Bool :=
  fun s =>
    let r := Layout5.setLeaf p x (toState s)
    (ofState r.1, r.2)
```

Every Lean quotation in this paper is a line range of the library rather than a transcription: the 19 panels here are generated from the source and re-checked against it, so a range that has drifted from the declaration it quotes fails the check rather than misquoting it silently.

4 Ownership semantics leads to reference count one

The discipline is a statement about authority: who may write, and who may hold a read. The count that decides which of the two implementations of a write runs is a count of references, and the bridge between the two is a theorem of the library's counted model. A store typed at an environment that counts its subject once, at a configuration in the correspondence, is owned by the account and its subject's reference is counted once: the typing's count of a *name* becomes the account's count of a *reference*.

The bridge *Scidonia/RC/Correspondence.lean, lines 305–307*

```
theorem bridge {ρ : TEnv} {m : Multi} {Γ : Occs} {y : Var} {r : Ref} {i : Nat} {v : Expr}
    (h : Corr ρ m Γ) (hlook : ρ.lookup y = some r) (hone : Γ.count y = 1) :
    Owns ρ m (.store (.var y) i v) ∧ (expand m).count r = 1 := by
```

At that count, the update is destructive: the written slot keeps its location and holds what was written, and the cell the store replaced is the cell it wrote. It is the branch the update takes at a count of one; whether the runtime's own exclusivity test agrees at that count is the hypothesis section 6 names.

Count one: the store keeps the location *Scidonia/RC/Ownership.lean, lines 559–564*

```
theorem store_owned_is_destructive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : (expand m).count r = 1) (hslot : (CHeap.slots c).count r = 0)
    (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks := by
```

At any other count it copies, and the theorem states the copy and not only the loss: the written slots land at a location the heap did not list, the old cell keeps its slots, and its count drops by one.

Any other count: the store copies *Scidonia/RC/Ownership.lean, lines 600–606*

```
theorem store_shared_copies {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi} {y : Var}
    {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : 2 ≤ (expand m).count r ∨ 1 ≤ (CHeap.slots c).count r) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup (fresh c ks) = some ks ∧
      (update c (hs ++ expand m) r ks).1.lookup r = CHeap.lookup c r ∧
      (update c (hs ++ expand m) r ks).1.count r = CHeap.count c r - 1 := by
```

So the chain in the model is: a sealed surface that cannot hand out an alias, a discipline that reaches the write with the object unshared, a bridge that turns the account's count of the subject's name into one of the subject's reference, and the destructive branch at that count. Section 6 says what that chain does and does not say about the machine.

5 It actually works: the examples

Three updates ship with the library, and each is an instance of the same discipline rather than an arrangement of its own. The first is the flat array: the primitive's own write, with no discipline around it; the updates below reduce to it. An index past the end is a caller error rather than a no-op — compiled, `Array.set!` panics on it — and the examples write only to slots the array has.

The flat array *ScidoniaExamples/Updates.lean, lines 76–77*

```
def arrSet (a : Array UInt64) (i : Nat) (v : UInt64) : Array UInt64 :=
  a.set! i v
```

The second is a nested array. The type is the layout's state type definitionally, so the instance's `rep` is `rfl`, and the cast a sealed write performs erases; the path is two indices, and the take-out, the checks and the put-back at both levels are the library's.

The nested array *ScidoniaExamples/Updates.lean, lines 98–100*

```
def nestedSet (a : Array (Array UInt64)) (i j : Nat) (v : UInt64) :
    Array (Array UInt64) × Bool :=
  SealedStateVar.setLeaf (σ := Array (Array UInt64)) (nestedPath i j) v a
```

The third is a family of shapes rather than one shape: the full binary tree of structs at depth k, whose layout is a recursion and whose state type is a nested pair. A write at a leaf walks k struct cells and rewrites one word.

The struct tree *ScidoniaExamples/Updates.lean, lines 121–123*

```
def structTree : Nat → Layout5.Layout
  | 0 => .leaf .uint
  | k + 1 => .struct (structTree k) (structTree k)
```

A write at a leaf *ScidoniaExamples/Updates.lean, lines 194–196*

```
def treeSet (k : Nat) (v : UInt64) (t : Layout5.State (structTree k)) :
    Layout5.State (structTree k) × Bool :=
  SealedStateVar.setLeaf (σ := Layout5.State (structTree k)) (treePath k) v t
```

Compiled, the array's two shapes differ by the whole buffer, and the harness measures both in one program. The exclusive shape carries the array as a recursive call's parameter, so the only reference the update sees is the callee's; the aliased shape holds a live second reference across each update, so each one replaces the buffer. Before the timed runs, each shape is probed once and the address the update kept or moved is reported beside the times.

The benchmark *ScidoniaInPlace/Main.lean, lines 1805–1816*

```
def runBenchmark (seed : Nat) : IO Unit := do
  IO.println "bench: the array update, exclusive (the caller alone refers to the array)
  ↪  against aliased (a live second reference)"
  for n in benchSizes do
    reportArrayBench (← benchArrayRow n seed)
  IO.println "bench: the tree update, exclusive against aliased; the walk is in both columns,
  ↪  so the extra is the allocation of the cells the update rebuilt"
  for k in benchDepths do
    reportTreeBench (← benchTreeRow k seed.toUInt64)
  IO.println "bench: the union-find, exclusive against aliased; a union's writes are take-out
  ↪  updates of the two arrays, and the aliased shape copies a buffer at every step"
  for n in ufBenchSizes do
    reportUfBench (← benchUfRow n)
  IO.println "bench: per-operation times are means over arrayOps (treeOps, ufOps) updates and
  ↪  arrayReps (treeReps, ufReps) measurements; a measurement of this program on this
  ↪  toolchain, not a complexity theorem"
```

Measured. The two tables below are one run of `nix develop -c lake exe inplace` on the machine used here. The times are means over 100 updates and 5 timed measurements per row for the array, and over 200 updates and 11 measurements per row for the tree; each row's untimed probes reported what the columns claim, the exclusive shape keeping the buffer's address and the aliased shape moving it. Two runs of the harness on this machine differ in the per-operation times — most of all in the tree's extra column, which is a difference of two larger numbers — so these numerals are one run's, and they are read for the shape of the columns rather than for the last digit.

Table 1: The array update, measured: nanoseconds per update at four sizes. The times are a measurement of this program on this toolchain, not a theorem, and the ratio is the ratio of the two measured means.

n	exclusive (ns/op)	aliased (ns/op)	ratio
1 000	30	1 227	40.9
10 000	30	15 285	509.5
100 000	41	218 500	5 329.2
1 000 000	41	3 601 979	87 853.1

The array's aliased column is the copy: it grows with the buffer, from about a microsecond per update at a thousand elements to 3.6 milliseconds at a million, while the exclusive column is flat across the four sizes, at tens of nanoseconds, and the last row's ratio is above eighty thousand.

Table 2: The tree update, measured: nanoseconds per update at four depths, from the same run and over the same means as table 1. The walk is in both columns, so the extra column — the difference of the two — is the allocation of the one cell per level the update rebuilds. Measured, not a theorem.

depth	leaves	exclusive (ns/op)	aliased (ns/op)	extra (ns/op)
8	256	589	668	79
12	4 096	1 194	1 246	52
16	65 536	2 141	2 197	56
20	1 048 576	2 743	3 003	260

The tree's two columns both grow with the depth, because each update walks the spine either way, and the extra column isolates the allocation: tens to a few hundred nanoseconds per update. Neither column is a constant-time operation, and nothing in these four rows separates the walk from the allocation by a theorem; that separation is in the model, as the allocation of the spine, and section 6 states it as the scoped claim it is.

The three updates above are one write each. A fourth example is an algorithm rather than an update, a union-find, and it is the canonical place where updating in place rather than copying is the whole point of the program: a union's amortized cost is near-constant because the two arrays are updated where they lie, and a copy on every write destroys that. Its arrays are flat arrays of machine words, sealed the same way the nested array is — the layout `.arr (.leaf .uint)` has that type as its state type, so the instance's `rep` is `rfl` and the cast a sealed write performs erases:

The union-find's arrays, sealed *ScidoniaExamples/UnionFind.lean, lines 64–68*

```
abbrev sealedWords : Layout5.Layout := .arr (.leaf .uint)

instance : SealedStateVar (Array UInt64) := ⟨sealedWords, rfl⟩
```

Each write is therefore the class's take-out update at a one-step path rather than a bare `Array.set!` — the library's owned exchange, its two checks and its put-back, with no `Layout5` written by hand:

One slot of a machine-word array *ScidoniaExamples/UnionFind.lean, lines 83–84*

```
def writeWord (a : Array UInt64) (i : Nat) (v : UInt64) : Array UInt64 :=
  (SealedStateVar.setLeaf (σ := Array UInt64) (wordPath i) v a).1
```

`find` walks from an element to its root and halves the path on the way, so the walk itself writes; `union` finds both roots and links the lower-ranked one under the higher, which is one write to `parent` and, at equal ranks, a second to `rank`:

The walk, halving the path *ScidoniaExamples/UnionFind.lean, lines 119–126*

```
def find : Nat → Array UInt64 → Nat → Array UInt64 × Nat
  | 0, parent, x => (parent, x)
  | fuel + 1, parent, x =>
    let px := parentOf parent x
    if px = x then (parent, x)
    else
      let gx := parentOf parent px
      find fuel (writeWord parent x gx.toUInt64) gx
```

The link, by rank *ScidoniaExamples/UnionFind.lean, lines 137–147*

```
def union (fuel : Nat) (parent rank : Array UInt64) (x y : Nat) :
    Array UInt64 × Array UInt64 :=
  let (parent₁, rx) := find fuel parent x
  let (parent₂, ry) := find fuel parent₁ y
  if rx = ry then (parent₂, rank)
  else
    let rankX := rankOf rank rx
    let rankY := rankOf rank ry
    if rankX < rankY then (writeWord parent₂ rx ry.toUInt64, rank)
    else if rankY < rankX then (writeWord parent₂ ry rx.toUInt64, rank)
    else (writeWord parent₂ ry rx.toUInt64, writeWord rank rx (rankX + 1).toUInt64)
```

Two runs perform the same unions of the same fixed sequence on the same two arrays, and they differ in one thing: what else refers to those arrays while a step's writes run. The disciplined run carries the arrays as the recursion's parameters and holds nothing else; the undisciplined run takes a second reference to both at every step and reads it after that step's writes, which is what makes the step's first write meet a buffer another name reaches:

The undisciplined shape *ScidoniaInPlace/Main.lean, lines 1298–1310*

```
def unionStepsAliased : Nat → List (Nat × Nat) → Array UInt64 → Array UInt64 →
    Nat → IO (Array UInt64 × Array UInt64 × Nat)
  | _, [], parent, rank, total => return (parent, rank, total)
  | fuel, (x, y) :: rest, parent, rank, total => do
    let aliasParent := parent
    let aliasRank := rank
    let (parent', rank') := union fuel parent rank x y
    let parentHere ← addressIO aliasParent
    let rankHere ← addressIO aliasRank
    let seen := ((aliasParent[x]?).map UInt64.toNat).getD 0
      + ((aliasRank[y]?).map UInt64.toNat).getD 0
    unionStepsAliased fuel rest parent' rank'
      (total + parentHere.toNat + rankHere.toNat + seen)
```

The two compute the same partition — the semantics does not change, only the cost — and that is checked rather than assumed: `ScidoniaTest/UnionFind` evaluates both runs on eight elements and four unions and compares the representatives `find` reports, asserts that every reported representative is its own parent, and states the sealed class's read-back law and its frame law at the algorithm's write path.

Measured. The table below is a run of the same harness on the same machine as the two above, and a run of its own: the array and tree tables were taken before this algorithm's rows existed, so the three are three runs, each read for the shape of its columns. The means and the probes are the same kind — `ufReps` measurements of `ufOps` unions per row, each shape probed once before the timed runs. The sequence's length grows with n, which is what makes a row's *total* readable as a shape as well as its per-union mean.

Table 3: The union-find, measured: nanoseconds per union and the mean total of the row's whole sequence, at four element counts. Measured, not a theorem; one run's numerals, read for the shape of the columns rather than for the last digit.

n	unions	excl. total (μs)	aliased total (μs)	excl. (ns/op)	aliased (ns/op)	ratio
256	64	17.7	83.9	277	1 310	4.7
1 024	256	67.8	1 195.2	264	4 668	17.6
4 096	1 024	280.5	28 568.9	273	27 899	102.1
16 384	4 096	1 013.9	298 075.4	247	72 772	294.6

The exclusive column is flat across the four sizes — 247 to 277 nanoseconds per union over a 64-fold range of n — and its total grows in step with the sequence, which is one near-constant

cost per union. The aliased column is the copy: 1 310 nanoseconds per union at $n = 256$ against 72 772 at $n = 16\,384$, growing with the buffer, so the totals diverge — 84 microseconds to 298 milliseconds, a factor of about 3 550 where n grew by a factor of 64 — and the ratio of the two per-union columns reaches 295. That is the contrast the example exists to show: the same algorithm, the same unions, the same partition, and one near-constant cost against a linear one, decided by nothing but what else refers to the arrays.

What the discipline gives this algorithm is the sealed take-out update for its writes and observation-bounded reads — the class's `read?` returns an element of the closed observation universe, never the array — and what it does not give is a proof that any of those writes went in place. That is measured, here and in section 6. The ownership typing the library develops elsewhere has first-order expressions as its subject and does not reach this Lean function: the union-find is written *with* the sealed class and is not checked by that typing.

Two checked children arm the in-place check on this path, and they stop at different guards. `uf-checked-shared` runs the aliased shape and must end the process naming the *array* site, which is the copy the step would take — the loud failure the other examples' children have, and the one the check exists for. `uf-checked-focus` runs the disciplined shape and stops too, at the *focus* site, and that is a fact about the check rather than about the update: the array is exclusively owned there, and the element the take-out extracted is a machine word — an element of an `Array UInt64` is a value and not a heap object — for which the guard's exclusivity test answers `false`. The same run with the check off settles it: `uf-unique` asserts that both buffers keep their addresses, so nothing copies. The array guard is therefore what this path can be armed against, and the in-place evidence for the exclusive run is that address assertion.

6 The honest boundary

Three seams separate what is proved here from what the machine does.

1. *The count is the model's.* Section 4 proves that a store at the model's count of one keeps its location, and that any other count copies. The proof is about the counted heap the library defines and the account the discipline produces; it is not a statement about Lean's object graph.

2. *The model's count is not the machine's test.* For a compiled write to be destructive, the runtime's own exclusivity test must accept the object, and the step from a model count of one to that test is a hypothesis the library names `RuntimeAgrees`. It is stated rather than assumed silently, and no instance of it is constructed, here or in the checks that accompany the library.

   The runtime agreement hypothesis *Scidonia/FracPerm/StateVar/Footprint.lean, lines 529–530*

   ```
   def RuntimeAgrees (h : Presentation) (handles : List Ref) (r : Ref) (rc : Int32) : Prop :=
     Exclusive h handles r → RefCount.IsExclusive rc
   ```

3. *In-place update is measured.* The evidence that a compiled write updates in place is that an executable observed the address the write kept, in the cases the harness names and in the probes behind table 1 and table 3. That is an observation of named cases on one toolchain, and no theorem here certifies it. The check that turns such a copy into a terminated process has a limit of its own, which the union-find's cases report: it tests the exclusivity of the array *and* of the element a take-out extracted, and a focused element that is a machine word rather than a heap object stops that test where no copy would happen, so a checked child on such a path stops at the focus site and the in-place evidence there is the address assertion.

The complexity claim is scoped the same way. For the tree, an update walks `depth` struct cells and rewrites one word in both columns, so the traversal is $O(\texttt{depth})$ however the write behaves; what ownership decides is the *allocation* of the spine — $O(1)$ cells rebuilt when the tree is exclusively owned, $O(\log n)$ when a live reference makes it shared. The result is not an $O(1)$ deep-tree operation.

One limit of the surface belongs beside these, because it bounds what a reader may conclude from section 3: a type joins the sealed class only when it is definitionally the layout's state type, since the class's `rep` is an equality and not an isomorphism. A user-declared structure would need the isomorphism instead, and what a conversion between two representations costs has neither been implemented nor measured.